COMP2271 - Computer Graphics

Demystifying the Black Box: The Rendering Pipeline

Dr. Frederick Li

Department of Computer Science, University of Durham

Dr. Frederick Li

Associate Professor in the Department of Computer Science

About me

- **Research Interests:** Computer Graphics, Computer Vision, Deep Learning, Educational Technologies.
- **Teaching:** L2 Computer Graphics, L3 Multimedia & Game Development, L4 Advanced Computer Graphics.
- **Administration:** Chair of UG Board of Examiners.

Contacts

- **Office:** MCS 1028 (Office Hour: Term 2, Thursday 9am–10am)
- **E-mail:** `frederick.li@durham.ac.uk`
- **Website:** `https://frederickli.webspace.durham.ac.uk/`
- **Google Scholar:** Profile Link (V9XyRX8AAAAJ)

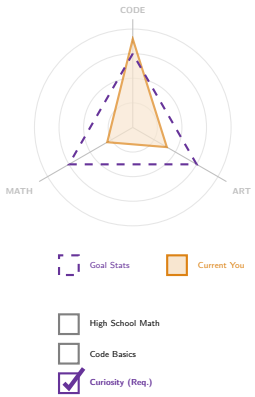
Player Select: Who Are You?

Calibrating the Difficulty Setting

The Pre-Flight Check

Turn to your neighbour (or shout out) to configure your profile:

1. **Origin Story**
   - Name & Degree Programme?
2. **Character Class**
   - **The Coder:** "I dream in C++ / Python / C# / OpenGL / WebGL / Three.js."
   - **The Artist:** "I know Blender / Photoshop."
   - **The Math Wizard:** "I love Matrices."
3. **The Quest**
   - Why this module? (Games? GPUs? Required?)



Status Update

"We are here to level up."

Lecture Schedule

Logistics, Locations & Timeline

This Sub-Module

- **Week 16 to Week 20:** Computer Graphics
- **Term 3:** One Revision Lecture

Time & Location

- **Thursdays:** 10:00 – 11:00 am @ **CG93**
- **Fridays:** 12:00 – 1:00 pm @ **CG93**

Session Activities

- Core CG Topics, Theory, Methods, CG Mathematics & Programming.
- Coursework Discussion & Support.
- Practical Sessions.

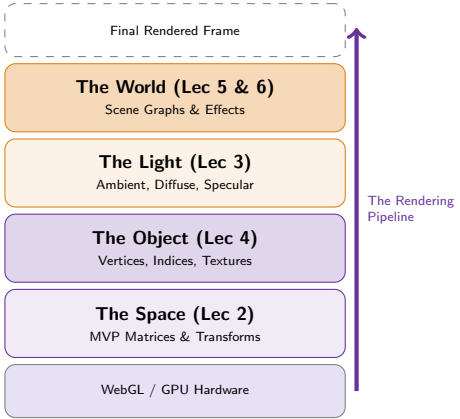
The Mission: Demystifying the Black Box

From Abstract Math to Virtual Worlds

The "Magic" View You call a function, and a 3D object appears. *(But how does a list of numbers become a picture?)*

The "Graphics" View We will peel back the layers of the API to understand the **Pipeline**.

- **Mathematics:** Mastering the **Matrices** and **Transformations** that move space (Lec 2).
- **Structure:** Organising chaos into **Scene Graphs** and **Meshes** (Lec 4 & 5).
- **Simulation:** Approximating physics with **Lighting Models** like Blinn-Phong (Lec 3).
- **Technique:** From standard Rasterisation to **Ray Tracing** logic (Lec 1 & 6).



The Curriculum: Your Toolset

From Vertices to Virtual Worlds

Phase 1: The Structure

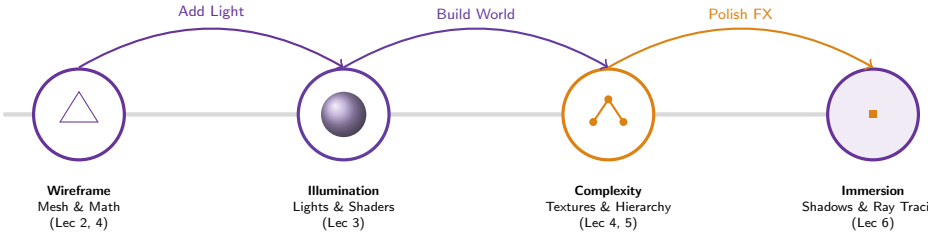
- **Lec 2: Transformations**
  Matrices, MVP Pipeline, Homogeneous Coords.
- **Lec 4: Geometry**
  Meshes, Vertices, VBOs & Indices.

Phase 2: The Appearance

- **Lec 3: Lighting**
  Blinn-Phong Model, Ambient/Diffuse/Specular.
- **Lec 4: Textures**
  UV Mapping, Sampling, Filtering.

Phase 3: The World

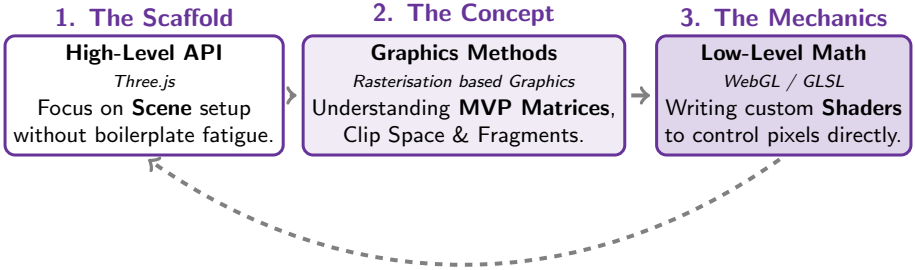
- **Lec 5: Scene Graphs**
  Hierarchies, Parent-Child Transforms.
- **Lec 6: Advanced FX**
  Ray Tracing, Post-Processing, Shadows.



How We Learn: The Scaffolding Method

Using Libraries as a Window, not a Wall

"We use the library to handle the boring parts (Boilerplate), so we can focus on the difficult parts (The Math & Pipeline)."



Abstraction
We use `Three.js` to get pixels on screen fast.

Dissection
We analyse how the GPU turns **Vertices** into **Fragments**.

Implementation
We define the **Mathematics** that drive the engine.

What is Computer Graphics?

The Art of Imitating Reality: Math, Physics, and Perception

1. The All-in-One Definition Computer Graphics is not just drawing pixels. It is the **computational simulation of reality**.

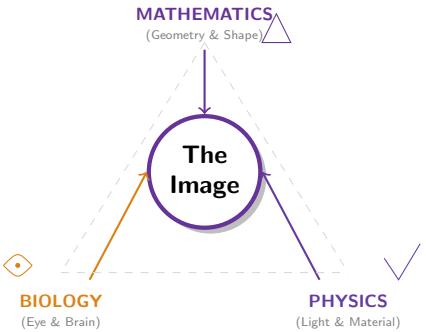
- **The Core Mechanism:** We use **Mathematics** (Matrices) to simulate **Physics** (Light transport) in order to deceive **Biology** (The Human Eye).

2. Why Now? The Modern Relevance

- **The Interface of the Future:** With VR/AR (Spatial Computing), the entire user interface is becoming a 3D graphics problem.
- **Digital Twins:** From manufacturing to climate science, we simulate the world to predict it.

3. The Insight: "Analysis by Synthesis" *"What I cannot create, I do not understand."* (Richard Feynman).

- CG forces us to reverse-engineer nature. To render a realistic face, you must understand skin subsurface scattering. To render water, you must understand fluid dynamics.



The Takeaway

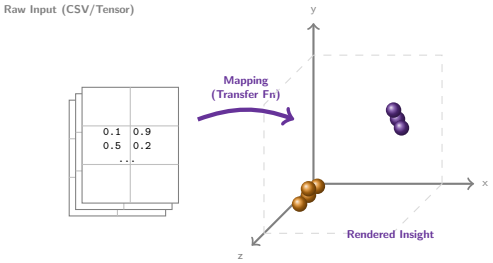
We do not just learn code.
We learn how the world works.

Why CG for Data Science? (1/2)

The Interface: Visualising High-Dimensional Data

1. Beyond the Spreadsheet Data Science is not just about calculating statistics; it is about *communicating* patterns.

- **Spatial Data:** Lidar scans, MRI volumes (Voxel data), and geospatial datasets cannot be understood as 2D arrays.
- **Dimensionality Reduction:** We often project high-dimensional data (e.g., t-SNE, PCA) into 3D space to find clusters.



2. The "Renderer" as a Tool To see the data, you must render it.

- **Volume Rendering:** Turning 3D arrays of density (medical data) into visible structure.
- **Interactive Dashboards:** WebGL drives modern data tools like Uber Kepler.gl.

The DS Connection

"If you cannot render it, you cannot explain it."

Why CG for Data Science? (2/2)

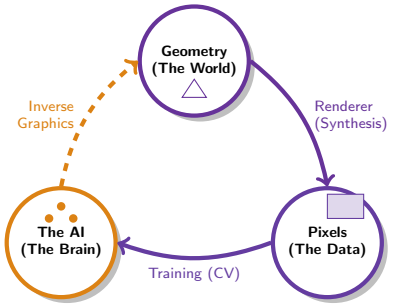
The Engine: Computer Vision & Deep Learning

Inverse Graphics & Synthesis Modern AI does not just categorise images; it tries to *understand* the world structure.

Modern Research Context In **Deep Learning & Computer Vision**, the lines are blurring:

- **Synthetic Data:** We can use CG to generate infinite, perfectly labelled training data for Neural Networks.
- **Analysis by Synthesis:** To detect a face, an AI effectively "renders" a face internally to match the input.
- **Neural Rendering (NeRFs):** AI can now replace traditional rendering pipelines.

Takeaway: *To build the next generation of AI, you must understand how light and geometry interact.*



The "Visual Loop"

CG generates data → AI learns → AI reconstructs 3D.

Expectation Management: The L2 Challenge

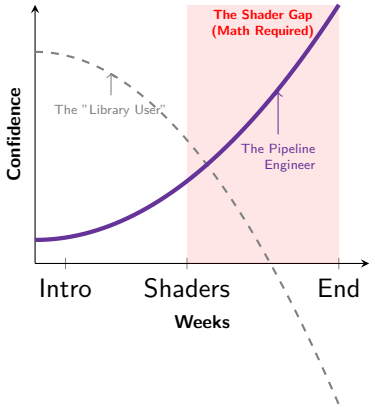
Moving from "User" to "Engineer"

1. The "Black Box" Illusion High-level libraries like Three.js are powerful, but they hide the complexity.

- **The Trap:** It is easy to copy-paste a tutorial and render a cube.
- **The Reality:** When lighting breaks, you cannot fix it without the **Pipeline** knowledge.

2. The Math is the Engine Graphics is just visual Mathematics.

- **Linear Algebra:** You cannot guess Matrix transformations ($T \cdot R \cdot S$).
- **The Shift:** Moving from *calling* functions to *writing* GPU logic.



The L2 Golden Rule

Don't just make it run.
Understand how it runs.

Teaching Approach & Your Journey

Building the Foundation, Not Just Passing the Exam

The Method: Theory ↔ Code

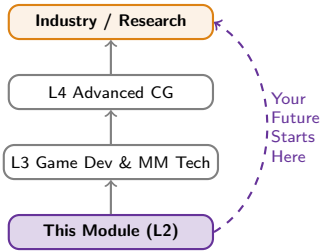
- **Interactive Lectures:** Live Demos connecting math theory directly to implementation (not just reading slides).
- **Supported Practicals:**
  - Programming exercises for muscle memory.
  - **Exam-style questions** for revision.
  - *Support:* Demonstrators have specific guides provided by me to help you unblock quickly.
- **Feedback Loop:** Model answers provided later: attempt first, check later.

Goal: Learn, Practice, Have Fun!

Alert: The Knowledge Chain

Graphics is cumulative.

- **Cramming Fails:** Memorising matrices last-minute does not work.
- **Engagement:** You must build the foundation week-by-week.



COMP2271 – 20-Credit Module

- Module: **Data Science**
- Faculty Handbook: `https://apps.dur.ac.uk/faculty.handbook/2025/UG/module/COMP2271`
- **Teaching Method:**

Activity	**Number**	**Frequency**	**Duration**	**Total/Hours**
Lectures	42	2 per week	1 hour	42
Practical Classes	19	1 per week	2 hours	38
Preparation and reading				**120**
Total				200

Component: Coursework		Component Weighting: 100%	
Element	Length / duration	Element Weighting	Resit Opportunity
Written Examination	2 hours	50%	Yes
Summative Assignment		50%	Yes

Assessment: Demonstrating Mastery

Proving Knowledge Without the Compiler

The Assessment of CG: Written Exam (25% of All COMP2271 Assessment)

Format: In-Person, Pen & Paper

- **No Safety Net:** No IDE, no compiler, no gen. AI in the exam hall.
- **The Challenge:** You must write shader logic and explain the *alignment* among code, mathematics and theories from memory.

The Trap: "The Code-Getter"

Using AI to bypass effort in Practical Exercises.

- **Action:** Asking AI to "Write this shader" and pasting it blindly.
- **The Gap:** You never learn *how* the syntax constructs the logic.
- **Exam Result: Failure.** You stare at a blank page.

The Strategy: "The Theory-Aligner"

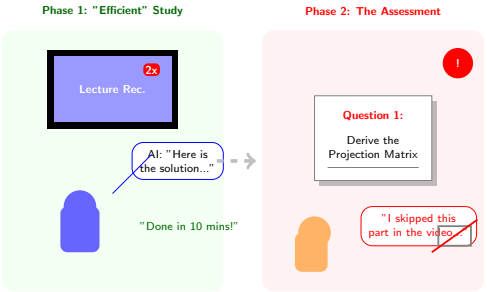
Using AI to dissect complexity.

- **Action:** Asking AI to "Explain why this Matrix is 4x4" or "Debug my logic."
- **The Gain:** You connect the code syntax to the Lecture Theory.
- **Exam Result: Success.** You understand the mechanics.

The Anti-Pattern: How to Fail L2 CG

Assessment: 100% Written Exam. No Coursework → No Place to Hide.

1. **The "Ghost" Strategy (Attendance)**
   - **Action:** Skipping physical lectures, assuming "I'll watch the video later."
   - **Reality:** You miss the **Interactive Clarification**. Physical attendance allows you to ask "Why?" when the concept gets hard. "Watching later" usually becomes "Skimming 2 days before the exam."
2. **The "Skimmer" Strategy (Study)**
   - **Action:** Watching recordings at **2x Speed** or fast-forwarding through the "boring theory parts" to see the cool visuals.
   - **Reality:** The exam tests the "boring theory parts." Deep understanding cannot be speed-run.
3. **The "AI Shortcut" Strategy (Practicals)**
   - **Action:** Using AI to *solve* your practical exercises instantly.
   - **Advice:** Use AI to *explain* a concept, but never to generate the solution.
   - **The Trap:** If AI solves the practical, you build no neural pathways. In the exam, you are on your own.



Interactive Lectures ≫ Skimmed Videos.
The exam requires deep derivation, not speed.

Let's Start the Journey

Initialising the Engine...

```
> System Check: READY
> Loading Module: COMP2271...
> Allocating VBOs/IBOs... [OK]
> Compiling Shaders (GLSL)... [OK]
> Calculating MVP Matrices... [OK]
> Initializing Lighting Models... [OK]

> WARNING: Math heavy ahead. Stay focused.

> Ready to enter the Pipeline? _
```

FPS: 60
Ghaz: High

EXECUTE:
`renderFrame(0);`

Let's make some pixels.